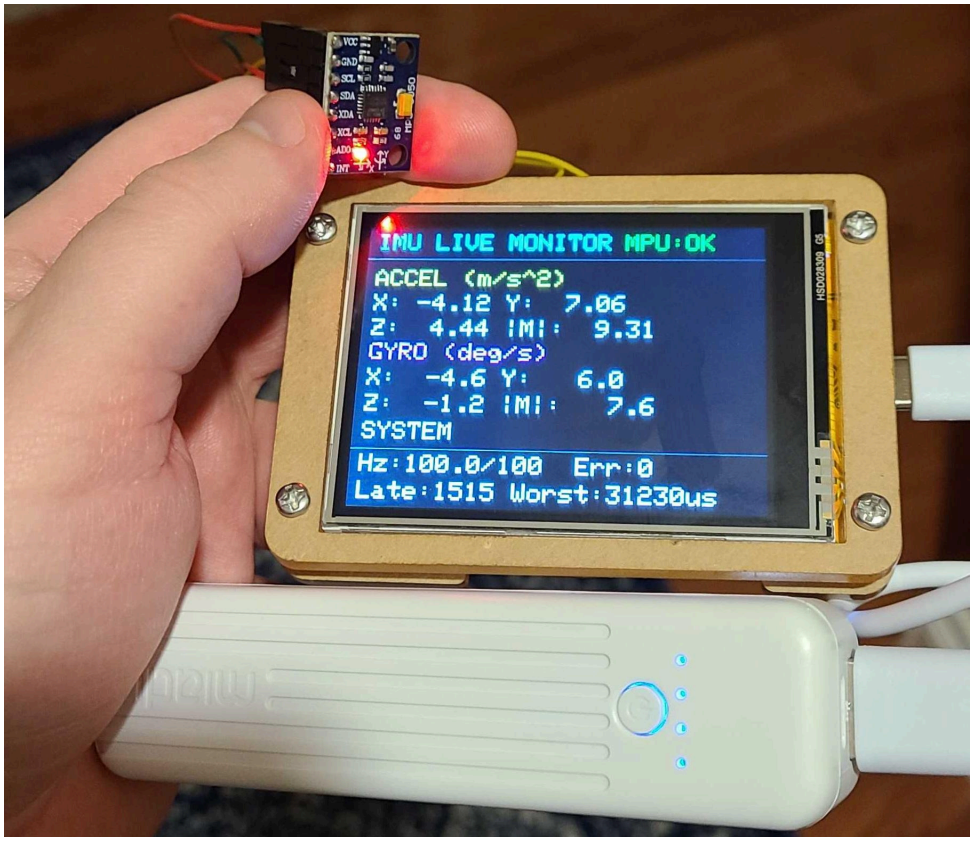
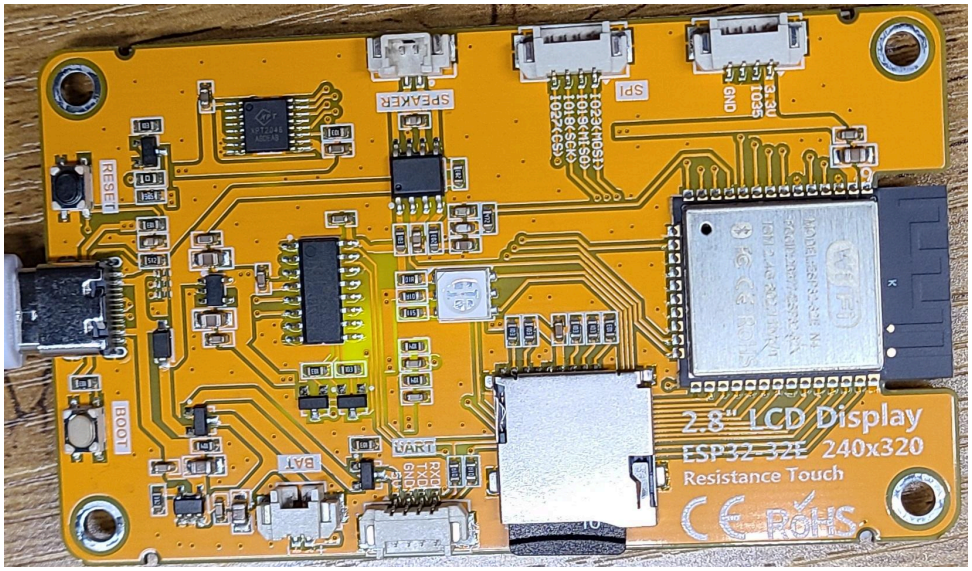
Prototype 0.2: ESP32-CYD (“Cheap Yellow Display”) + MPU-6050





Prototype Overview: This prototype was meant purely for learning purposes and has no functional purpose towards our research project and thus being given a “zero” designation as a result. The “Cheap Yellow Display” (CYD) unit was very cheap and features an ESP32 microcontroller that is pre-soldered into a small touch display. It also has a microSD card slot and wireless + bluetooth capability. It comes pre-loaded with a demo program that showcases the potential of the CYD environment.

Prototype Goal:

1. Learn how to flash program instructions in a “Bare metal” environment.
2. Learn how to interface with a basic IMU sensor (MPU-6050).
3. Learn how to display a 3d vector that moves in correlation with the attached sensor.
4. Learn how to write to the microSD card.
5. Learn how to transmit the data wirelessly either by wifi or bluetooth.

Goals met? Why or why not?

1. Yes. Arduino IDE was installed and I learned the basic environment and tools associated with it.
2. Yes. Data was successfully read from the MPU-6050. The numbers change as I move the chip around.
3. Yes. The vector displays properly and moves as I manipulate the free-floating MPU-6050. However, there is occasional drift. Correcting the drift is not within the scope of this learning prototype.
4. No. Please see the conclusion section for more details regarding why.
5. No. This device has a small amount of onboard flash memory and adding this connectivity is incredibly difficult in this environment. It would be much easier to learn this on a more capable device and then learn how to truncate and reduce the size of the code later.

Prototype Conclusion: The CYD is absolutely amazing as a learning device for basic concepts but it has some limitations to be aware of:

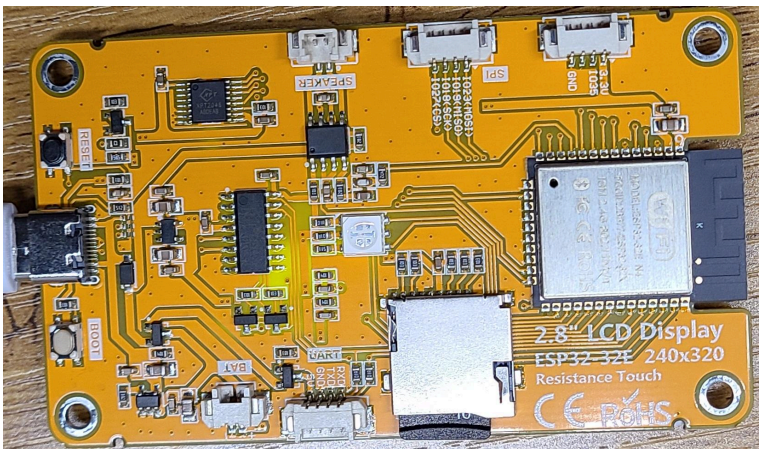
1. Low flash memory size. I had to use special settings that gave more physical room for the program at the expense of available ram. Adding lots of features in this environment feels largely impractical and would be more suited for advanced coders.
2. Low availability of code examples. It’s a bit difficult to find code examples for this device. I had to resort to scraping amazon reviews for working code examples.
3. Low amount of free ports for external devices. Most of the ESP32’s GPIOs are taken up by the touch screen which leaves you room for 1-2 sensors at most.
4. The microSD card slot shares a bus lane (VSPI) with the touchscreen’s touch input line. Code complexity increases greatly to be able to use both together. Proper usage means that the touchscreen has to be briefly disabled whenever starting a read / write task with the microSD card

I would grade this zero prototype as an overall success. Most of my goals were met and the ones that weren’t could be deferred to future prototype projects utilizing devices with higher capabilities.

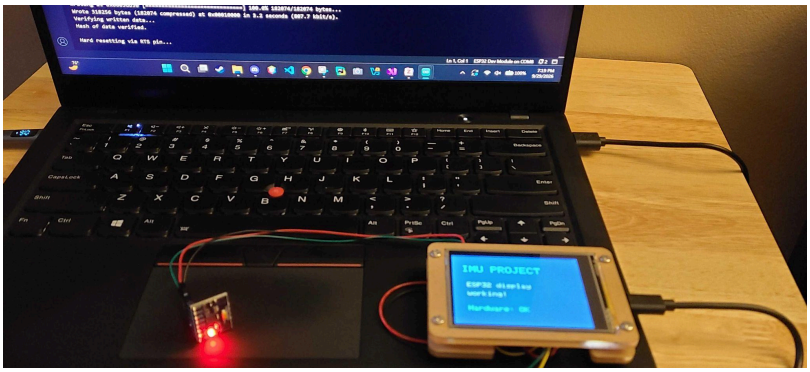
Pictures:



Caption: This is the Demo program that comes pre-loaded. There are 3 tabs up top and a CPU display bottom-right.



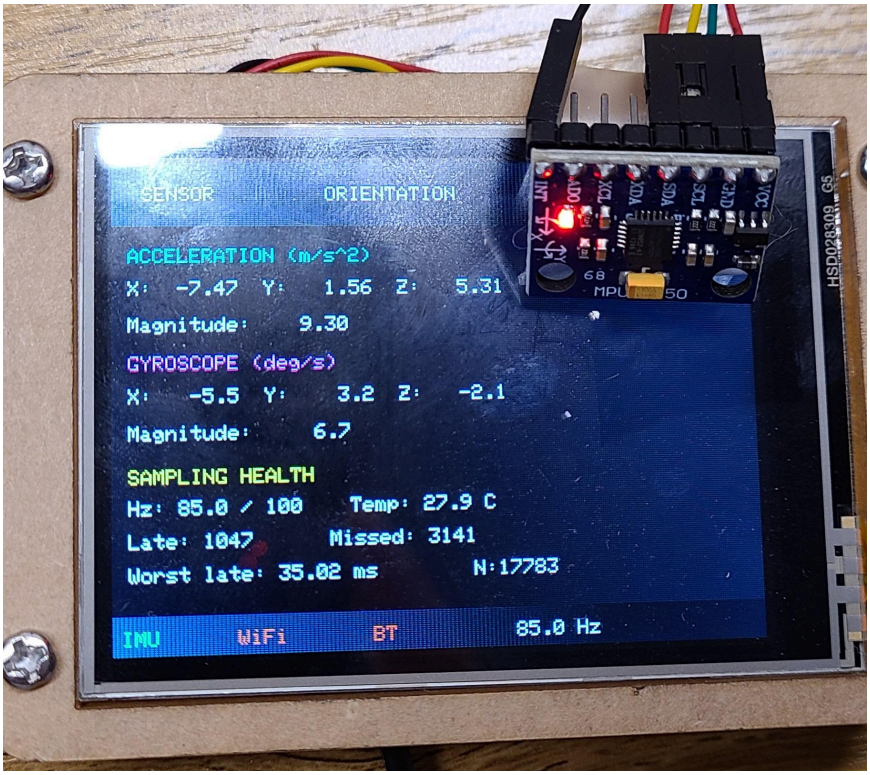
Caption: This is the bottom-side of the unit showing the traces and available ports. I utilized the SPI port (top-middle) as well as some pins from the adjacent misc port (top-right) to link in the MPU-6050.



Caption: Me using Arduino IDE to upload a primitive first sketch that just checks to see if it can detect the MPU-6050 sensor.



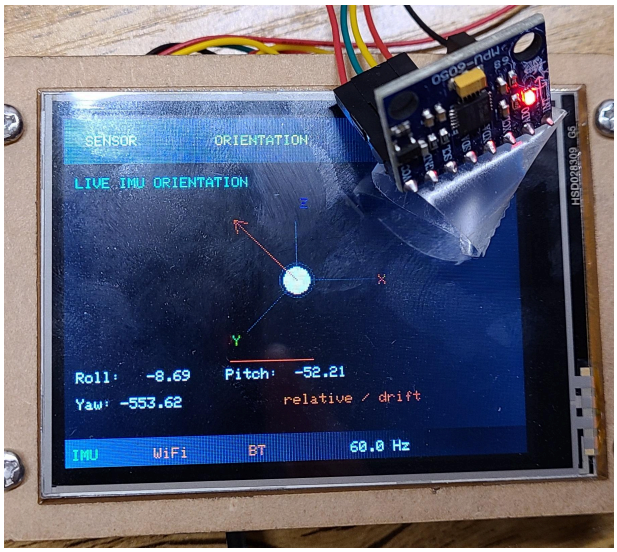
Caption: This is my next sketch that actually reads the x/y/z coords from the MPU-6050 and I have it powered from a powerbank.



Caption: Final sketch showing data being read from the sensor.



Caption: Final sketch showing the 3d vector depiction (vector is red and pointing almost directly down and out of frame)



Caption: Final sketch showing the 3d vector depiction with a different sensor orientation (red vector is pointing 45° diagonally up and left)